

## Lecture 2.2: Introduction to Regular Expressions

Presenter: Azwad Anjum Islam (AAI)

Scribe: Mujtahid Al-Islam Akon (AKO)

We know that there are four different types of automata. Each automaton has its own language that it recognizes/accepts:

| Automaton              | Corresponding language           |
|------------------------|----------------------------------|
| Finite State Automaton | Regular language (RE)            |
| Pushdown Automaton     | Context-free language (CFL)      |
| Linear Bound Automaton | Context sensitive language (CSL) |

In this lecture, we shall learn more about regular languages and how to express them algebraically in terms of regular expressions.

### Describing a Regular Language

**Regular language:** It is a simple language with a bunch of strings in it. We shall learn it more formally later.

- A simple regular language can be described intuitively in several ways.
  - You can list all the strings of the language in a tabular set format e.g.  $\{ac, ca, abc, cba, ac, ca, abc, cba, abdbc, cbbdda, cdda, adc, \dots\}$ . If the language is too large or infinite, this description is not good.
  - Also, you can start describing the language in words like we do in set builder method. We can express the above language as-
 

*{Containing strings that start with an **a** or a **c**.  
 If it starts with an **a**, then it ends with a **c**,  
 and if it starts with a **c** then it ends with an **a** ...}*

 You can see that the description can be too wordy sometimes but suitable for many simple cases.
  - You can draw an FSA that accepts the above language. Though this method will work perfectly, the drawing something is a bit tedious too. Is there a better way?
  - Lastly, we have Regular Expression which is easy to derive and often concise to write like a mathematical expression.
- Recall, often the strings in a language has something in common i.e. a common property. There is a pattern in all the strings of a language. For example-
  - $\{1, 10, 100, 1000, \dots\} \Rightarrow$  a set of strings starting with a **1** followed by 0 or more number of **0**'s
  - $\{a, aa, aaa, aaaa, \dots\} \Rightarrow$  a set of strings with any number of **a**
  - $\{1, 2, 3, 4, 5, 6, 7, \dots\} \Rightarrow$  a set of all the natural numbers.

Sometimes the pattern is not easy to find. For example-

- $\{are, republic, cream, brew, require, \dots\} \Rightarrow$  a set of strings having a substring **re**

- $\{\$a, locMin, \_num1, \_1204, \dots\} \Rightarrow$  valid java identifiers.
- Whether you can find it or not, there is a pattern in every regular language. This pattern is known as an **Expression**.
- If a language is regular, then its expression is called a regular expression (**RE**) or **RegEx**.
- Every regular language (**RL**) has one or more regular expressions, i.e. RE of a language is not unique.
- What is an RL actually? Before defining RLs, we need to know more about REs.

## The three basic regular operations

- In a regular expression, three types of operators can be present-

| Operator name             | Symbol           |
|---------------------------|------------------|
| OR Operator               | or +             |
| Concatenation Operator    | . or <no symbol> |
| Kleene closure operator   | *                |
| Positive closure operator | +                |

Don't be confused seeing the + sign in both OR and in the positive closure.

- In the positive closure, the + sign is a superscript i.e. it stays at the top of a symbol.
- In the OR operation, + sign sits between two symbols or two strings
- You have seen concatenation and the closure operators in the previous lecture.
- Some other operators may come as well, however, those are mainly derivatives of these three.
- **The OR operator:** It is a binary operator like what you saw in Boolean algebra.
  - **Position:** It sits between two things e.g.  $R_1 | R_2$  where each of  $R_1$  and  $R_2$  can be any string, a symbol or even another RegEx. You can also write them using + sign e.g.  $R_1 + R_2$ .
  - **Meaning:**  $R_1$  or  $R_2$  means that you may take the one from the left or the one from the right. If you take the one from the left, you get  $R_1$ , and if you take the one from the right, you get  $R_2$ .
  - **Examples:**
    - $a | b = \{a, b\}$
    - $aa | b = \{aa, b\}$
    - $hello | hi = \{hello, hi\}$
    - $(0 | 01) | 001 = 0 | 01 | 001 = \{0, 01, 001\}$
    - Etc.
- **The Concatenation operator:** It is a binary operator but it is **NOT** like the **and** operator that you saw in Boolean algebra.
  - **Position:** It sits between two things e.g.  $R_1 \cdot R_2$  where each of  $R_1$  and  $R_2$  can be any string, a symbol or even another RegEx. You can also write  $R_1$  and  $R_2$  side by side without any sign e.g.  $R_1 R_2$ .
  - **Meaning:**  $R_1$  or  $R_2$  means that you take the  $R_1$  first and then append  $R_2$  to the right.
  - **Examples:**
    - $a \cdot b = \{ab\}$
    - $aa \cdot b = \{aab\}$
    - $hello \cdot hi = \{hellohi\}$

- $(0 \cdot 01) \cdot 001 = 0 \cdot 01 \cdot 001 = \{001001\}$
- Etc.

**Note:** You cannot get more than one expression using only concatenation operators as many as you want.

- **Combination of Concatenation and OR**

- If you combine both, you may need to use the distributive law sometimes. For example-
  - $(a|b) \cdot c = \{ac, bc\}$
  - $(a|aa) \cdot (b|c) = \{ab, aab, ac, aac\}$
  - $hello \cdot hi \cdot (oh|no) \cdot bye = \{hellohiohbye, hellohinoby\}$
- Concatenation has higher precedence than OR. For example-
  - $a \cdot b|c$   
 $= (a \cdot b) | c$   
 $= \{ab, c\} \neq \{ac, bc\}$
  - $hello | hi | oh \cdot no | bye$   
 $= hello|hi|(oh \cdot no)|bye$   
 $= \{hello, hi, ohno, bye\}$

- **The Kleene Closure operator:** It is a unary operator i.e. it only works with one operand.

- **Position:** It sits top on something e.g.  $R^*$  where  $R$  can be any string, a symbol or even another RegEx.
- **Meaning:**  $R^*$  means **zero** or more occurrences of  $R$ .
- **Examples:**
  - $a^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$
  - $(hello)^* = \{\epsilon, hello, hellohello, hellohellohello, \dots\}$

- **The Positive Closure operator:** It is a unary operator i.e. it only works with one operand.

- **Position:** It sits top on something, e.g.  $R^+$  where  $R$  can be any string, a symbol or even another RegEx.
- **Meaning:**  $R^+$  means **one** or more occurrences of  $R$ .
- **Examples:**
  - $a^+ = \{a, aa, aaa, aaaa, \dots\}$
  - $(hello)^+ = \{hello, hellohello, hellohellohello, \dots\}$
  - Note that,  
 $\epsilon^+ = \{\epsilon, \epsilon\epsilon, \epsilon\epsilon\epsilon, \epsilon\epsilon\epsilon\epsilon, \dots\}$   
 $= \{\epsilon, \epsilon, \epsilon, \epsilon, \dots\}$   
 $= \{\epsilon\}$

- Kleene or positive closure has the **highest precedence** of them all, then the Concatenation operator and lastly the OR operator. For example-

- $(a|b^*) \cdot c = (a|(b^*)) \cdot c = \{ac, c, bc, bbc, bbbc, \dots\}$
- $ab | (xy)^* = \{ab, \epsilon, xy, xyxy, xyxyxy, \dots\}$
- $(01)^* = \{\epsilon, 01, 0101, 010101, \dots\}$
- $(0^*|1^*) = \{\epsilon, 0, 00, 000, \dots, 1, 11, 111, \dots\}$
- $(a|b)^* = (a|b) \cdot (a|b) \cdot (a|b) \cdot (a|b) \cdot \dots$   
 $= \{\epsilon, \text{all the strings made with } a \text{ and } b\}$

Informally, any expression that you can create using OR ( $|$ ), Concatenation ( $\cdot$ ), and the Closure ( $^*$ ,  $^+$ ) operators are called regular expressions.

## Equivalence of regular expressions and regular languages

There are two conditions for an expression to be considered as a regular expression of a certain regular language. This conditions as a whole is known as the equivalence of regular expressions and regular language. They are as follows-

1. Any string that matches with the regular expression has to be a member of the regular language.
2. Every string of the language must match with the regular expression.

Thus, the following cases hold-

- If a string matches with the regular expression  $R$ , but is not a member of the regular language  $L$ , then,  $R$  is not the regular expression of  $L$ .
  - Again, if there is a member of  $L$  that does not match with  $R$ , then  $R$  is not the expression of  $L$ .
-